

JC4004 – Computational Intelligence

Sessions 33 & 34: Hybrid models

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)

Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)

Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

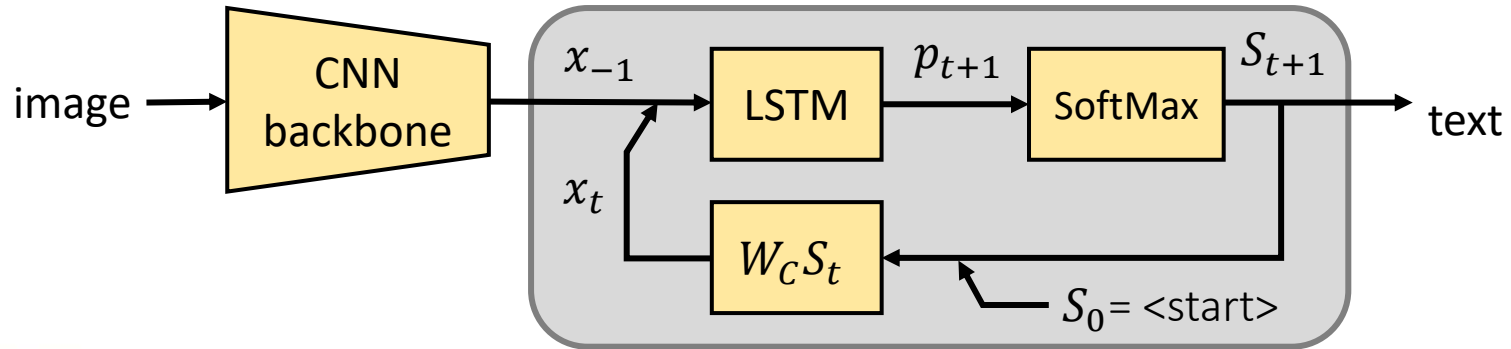
Oct-Dec 2025

Motivation for hybrid models

- Earlier in this course, we have considered CNNs, RNNs, and transformers as separate (and maybe competing) types of networks
- However, each technique have their strengths and weaknesses, and in many applications, it can be reasonable to combine them
 - CNN and RNN for analysing sequences of images (video in particular!)
 - Hybrid models of CNNs and transformers have been explored in many applications, especially in computer vision
- In today's sessions, we will introduce some hybrid model architectures
 - However, note that many others have been proposed as well!

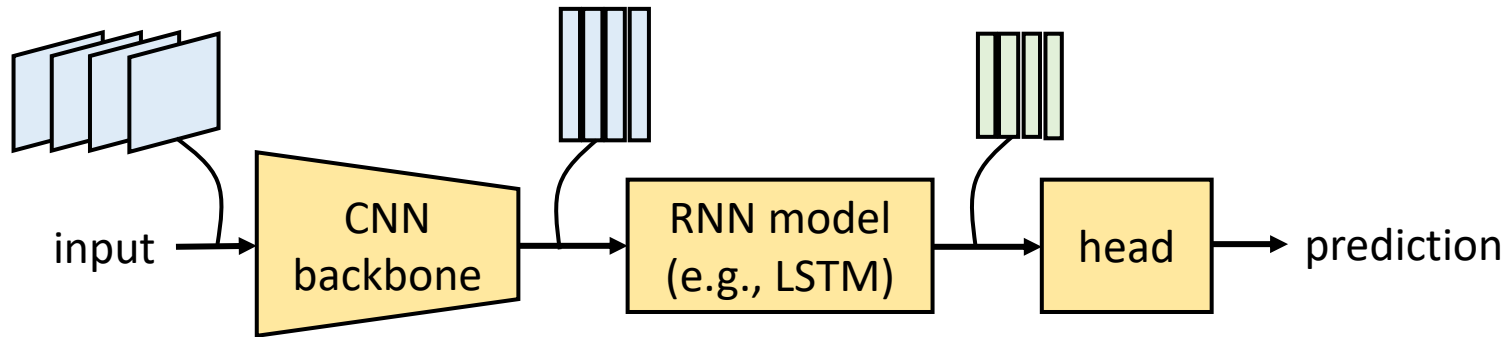
Image captioning with CNN-LSTM models

- Before transformers, RNNs were commonly used for text generation
 - An obvious idea is to use CNN backbone to extract image features and use the feature vector to initialise the LSTM generating image caption
 - LSTM can be used to generate a sequence of vectors $p = \{p_1, \dots, p_n\}$ representing probability distribution over the vocabulary



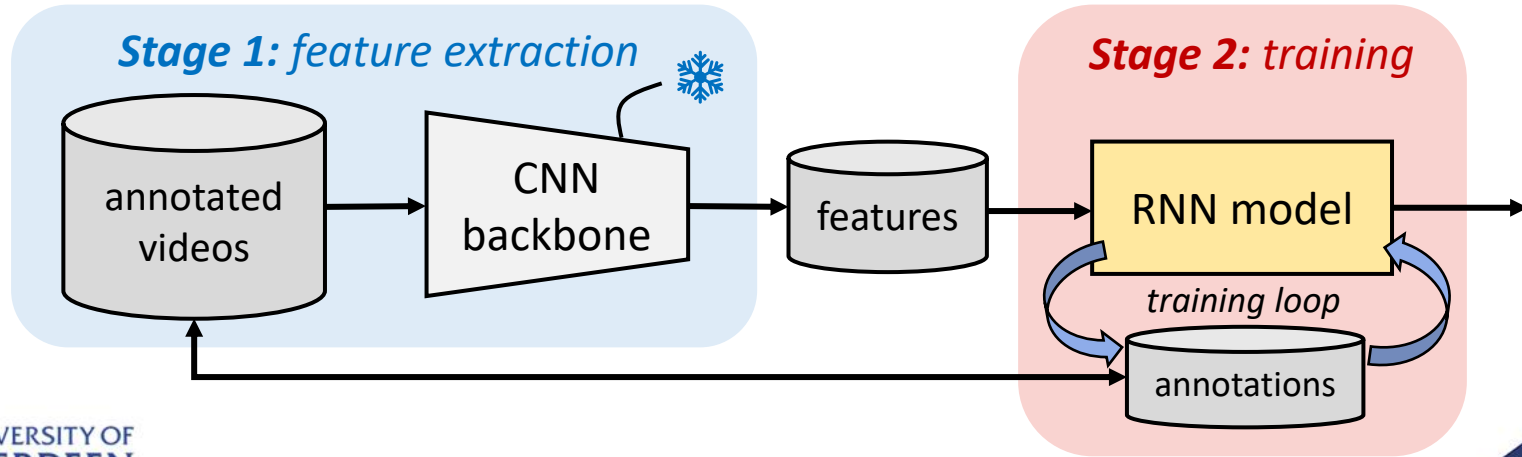
Hybrid CNN-RNN models for sequences

- CNN is efficient for feature extraction (especially in computer vision), whereas RNN is efficient for time series analysis
 - It is natural to use CNN as a preprocessor or feature extractor to generate a sequence of feature vectors (or tokens) and pass them to RNN



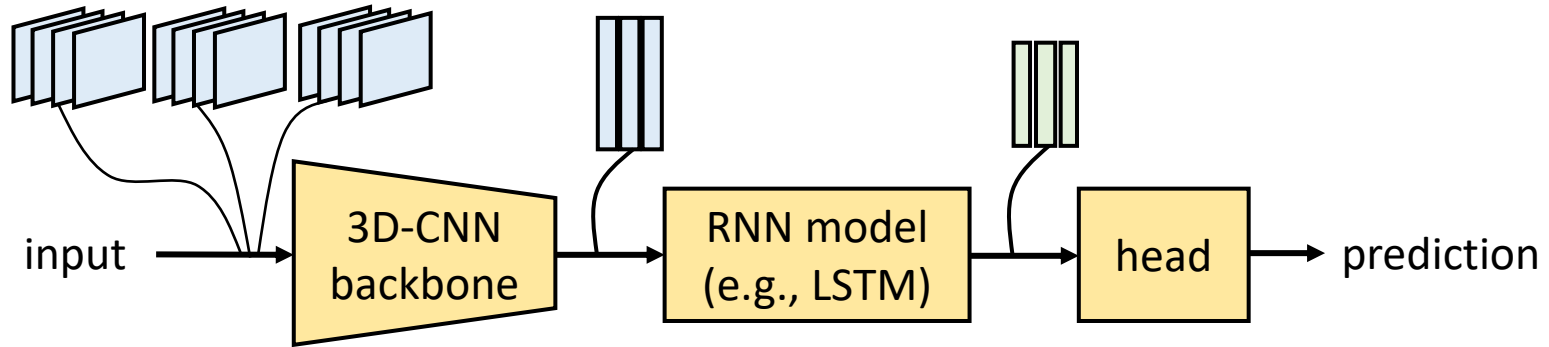
Training hybrid video classification model

- The basic approach for training CNN-RNN video classification model is to extract video features with CNN first and then train the RNN model
 - Pre-trained CNN backbone with fixed weights used for feature extraction
 - Saved feature vector sequences used for training the RNN model



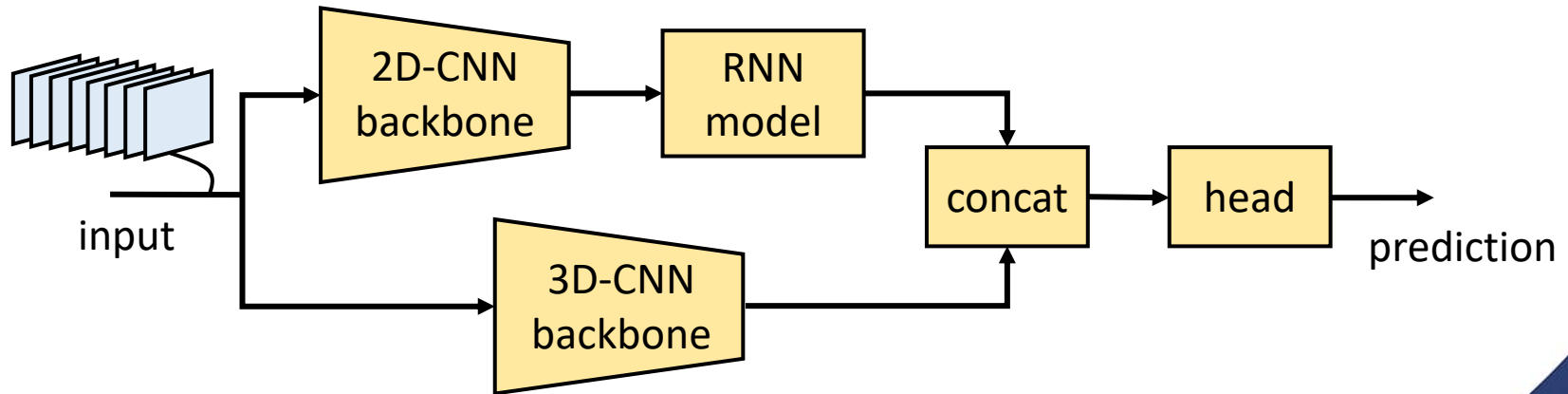
Hybrid 3D-CNN and RNN models

- One possible technique is to use 3D-CNN to split input video into short sequences, apply 3D-CNN to extract features for the short sequences, and finally use RNN to analyse the full video sequence
 - Especially useful for classifying long videos



Hybrid 3D-CNN and 2D-CNN-RNN models

- 3D-CNN and 2D-CNN-RNN are often seen as competing techniques, but they could also complement each other
 - Two techniques in parallel can be more accurate than the techniques alone

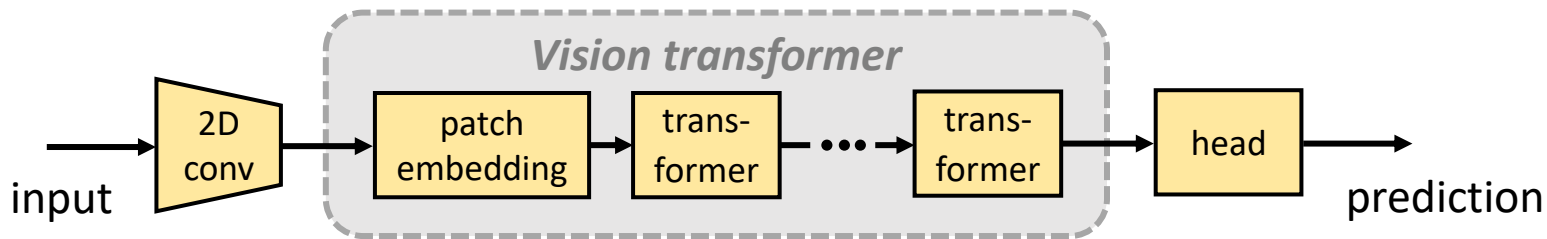


Hybrid CNN-transformer models

- In the past few years, transformers have replaced CNNs in many practical applications
 - Transformers are efficient in processing tokens with a semantic meaning
- However, there is a common understanding that CNNs can extract more meaningful low-level features than transformers
 - So, why not to use CNN for low-level feature extraction and transformers for processing the tokens with high-level information?
 - Especially in computer vision, there are several proposals to employ CNNs and transformers jointly for various tasks

Early CNN-transformer hybrids

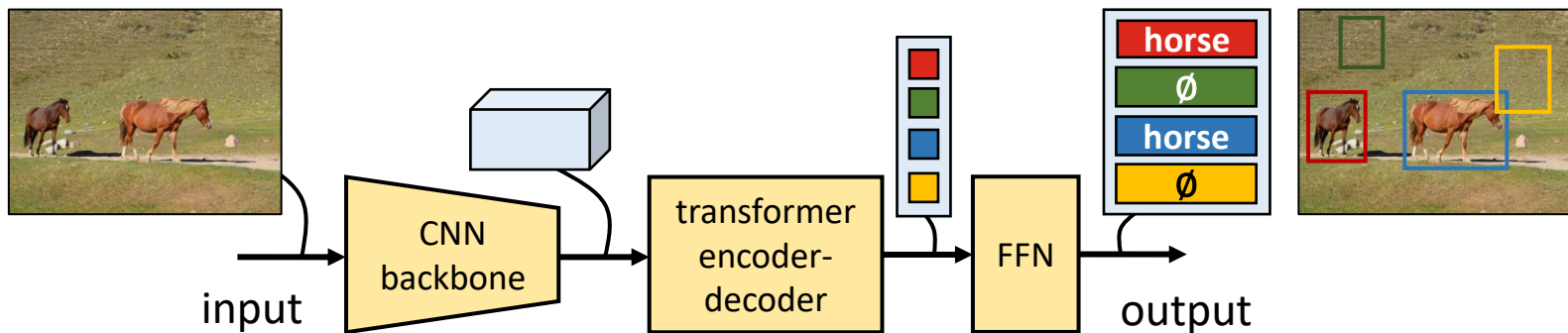
- The original ViT paper stated that they experimented ViT on top of CNN base layer in addition to pure transformer model
 - The output of convolutional layer used as input to ViT
 - Performance reported to be similar with pure transformer model



- Already before ViT, CNNs and transformers had been combined for object detection in *Detection transformer (DETR)*

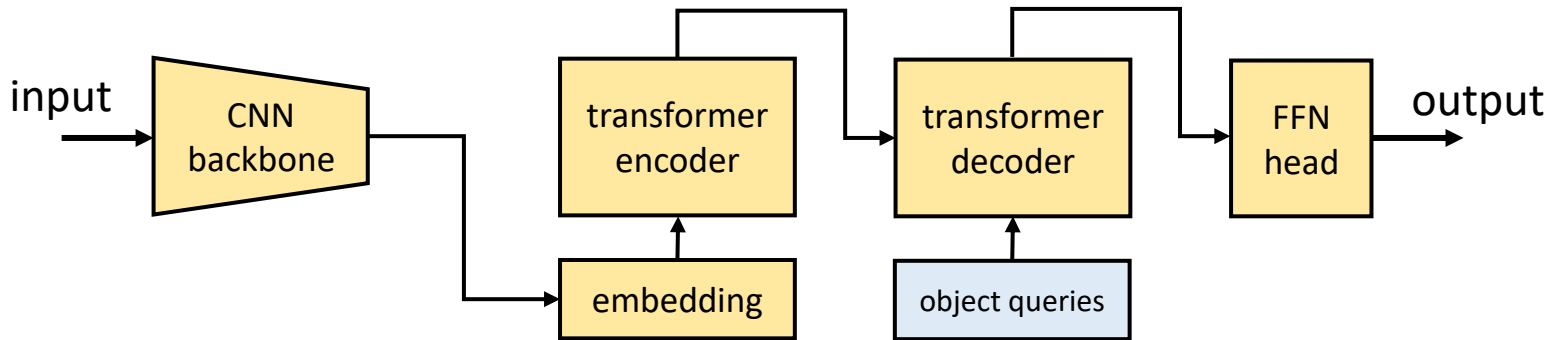
Detection transformer (DETR)

- *Detection transformer (DETR)* is a model for object detection
 - Classic convolutional backbone network extracts image features
 - Transformer encoder-decoder predicts a fixed set of potential bounding boxes for objects located in the image
 - Prediction head (FFN) predicts the class and coordinates of each box



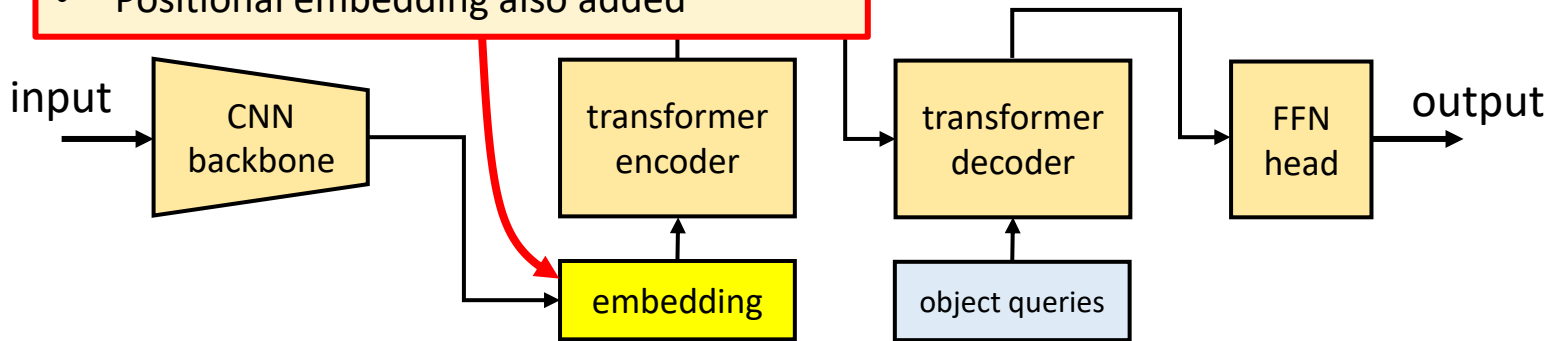
DETR architecture

- DETR transformer encoder-decoder has one-pass structure
 - DETR encoder part takes low resolution feature map from the CNN as input (e.g., 2048 channels, resolution $H = H_0/32, W = W_0/32$)
 - DETR decoder initialised with learned *object queries* as input and generates a fixed set of tokens representing potential bounding boxes



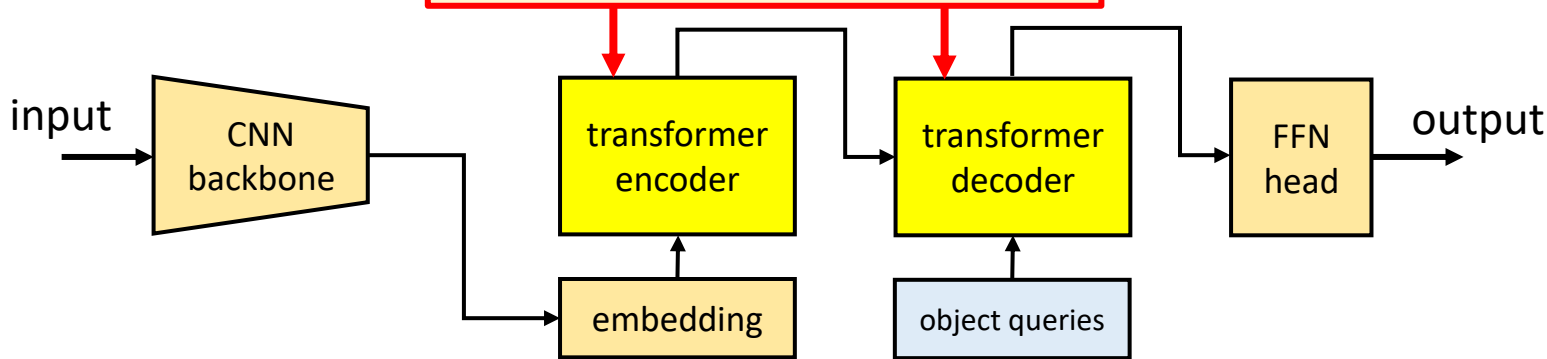
DETR architecture

- DETR transformer encoder-decoder has one-pass structure
 - DETR encoder part takes low resolution feature map from the CNN as input (e.g., 2048 channels, resolution $H = H_0/32, W = W_0/32$)
 - Each spatial position in the $H \times W \times C$ matrix converted to a token with depth C
 - Positional embedding also added



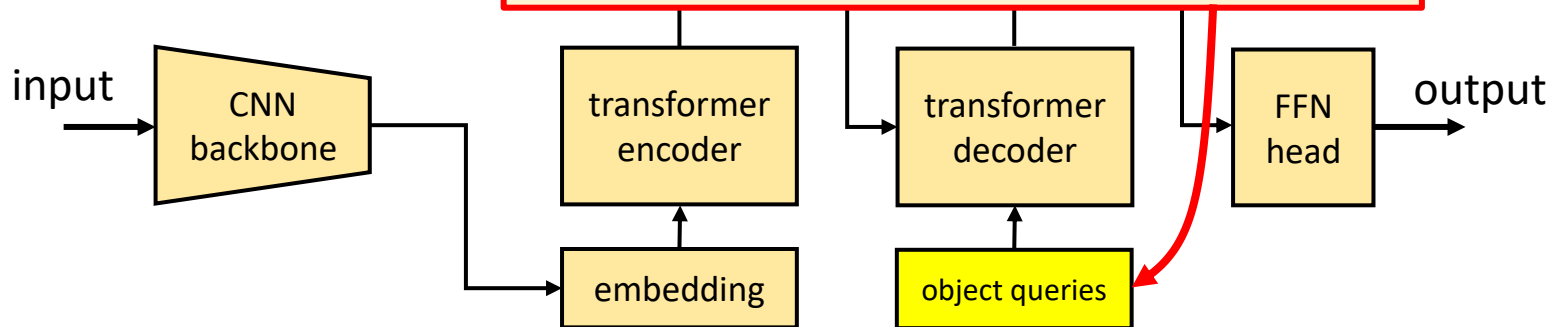
DETR architecture

- DETR transformer encoder-decoder has one-pass structure
 - DETR encoder part takes low resolution feature map from the CNN as input (e.g., 2048 $\times$ 2048 $\times$ 3, $H = H_0/32$, $W = W_0/32$)
 - DETR decoder input is the output of the encoder and generates a fixed number of object queries



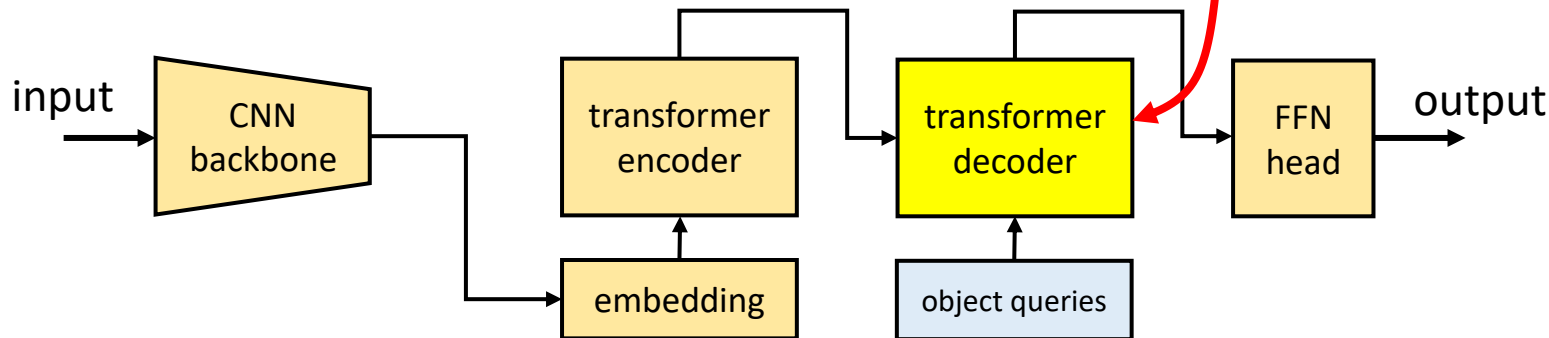
DETR architecture

- DETR transformer encoder-decoder has one-pass structure
 - DETR encoder part takes input (e.g., 2048 channels)
 - DETR decoder initial state generates a fixed sequence of object queries
 - Unlike the original transformer decoder, DETR decoder computes all the outputs at once
 - For this purpose, the decoder takes a fixed number of learnable positional embeddings (object queries) as input



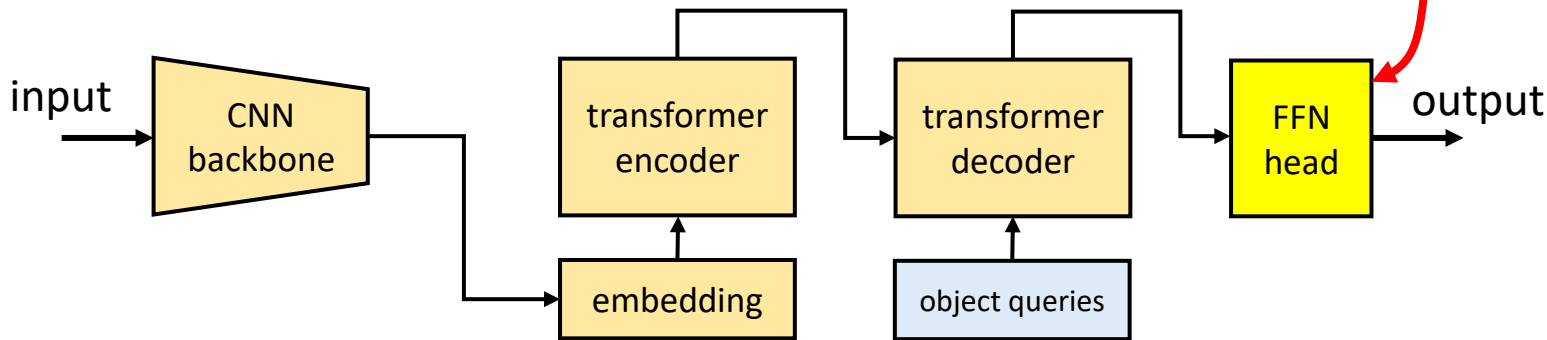
DETR architecture

- DETR transformer
 - DETR encoder processes input (e.g., 2048)
 - DETR decoder generates a fixed number of output vectors
- DETR decoder generates as many output vectors as there are object queries, each representing one bounding box
- There are more object queries than usually objects, so one vector can also represent a box with no object



DETR architecture

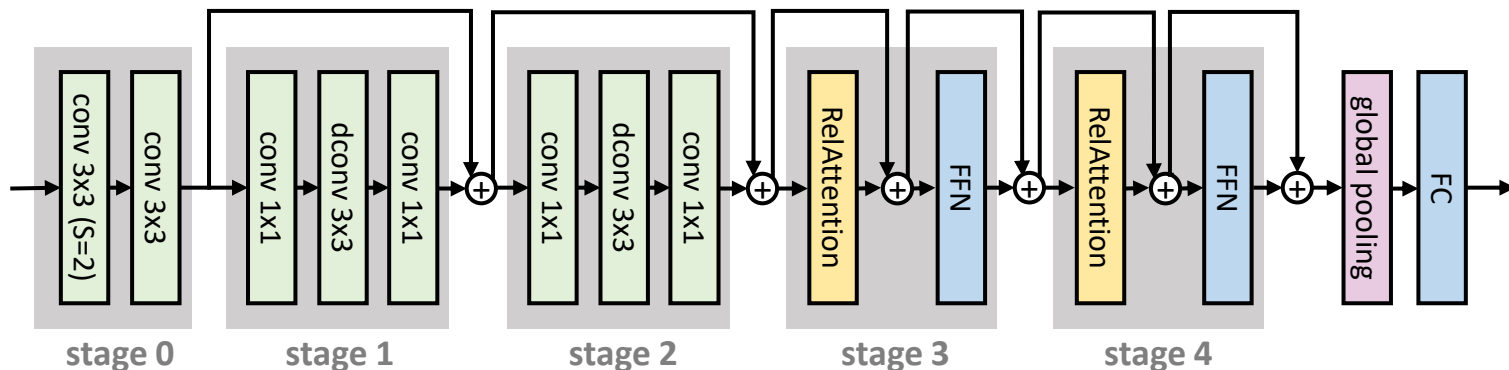
- DETR transformer encoder-decoder has one-pass structure
 - DETR encoder part takes low resolution feature map from the CNN as input (e.g., 2048 channels)
 - DETR decoder initially generates a fixed set of object queries
 - Feed-forward network applied to obtain the final bounding box coordinates and the respective classes (or “no class”) from the output tokens



Break: any questions?

CoAtNet architecture

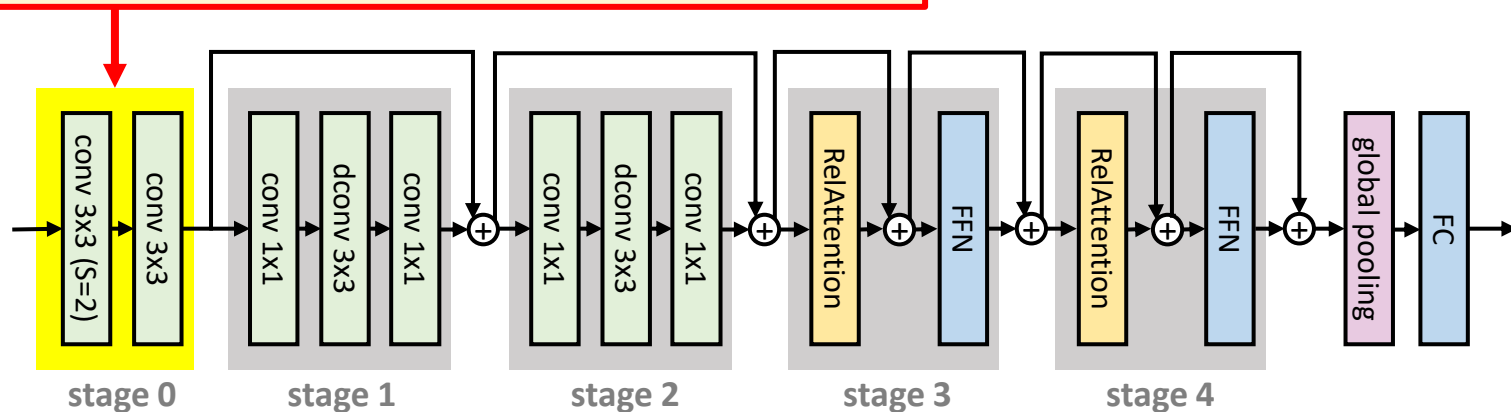
- Dai et al. (2021) studied combinations of convolutions and attention, and proposed a family of CoAtNet models
 - Three convolution stages and two transformer stages with different number of blocks for different versions of the model



CoAtNet architecture

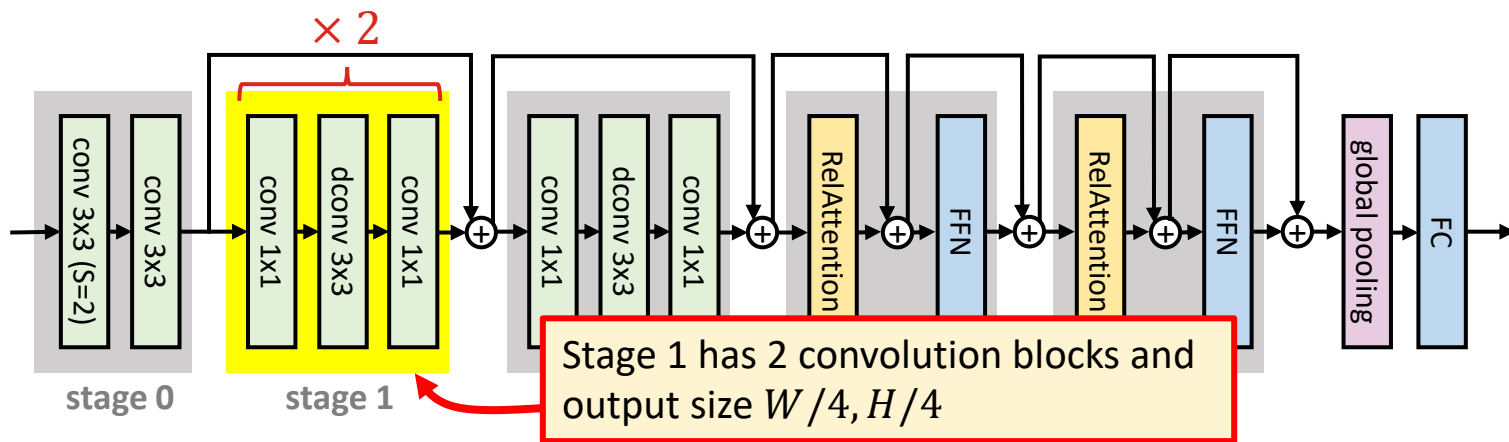
- Dai et al. (2021) studied different CNN-transformer hybrid configurations and proposed a family of CoAtNet models

Stage 0 is the fixed stem stage with two 3x3 convolution operations, output size is $W/2, H/2$ of the model



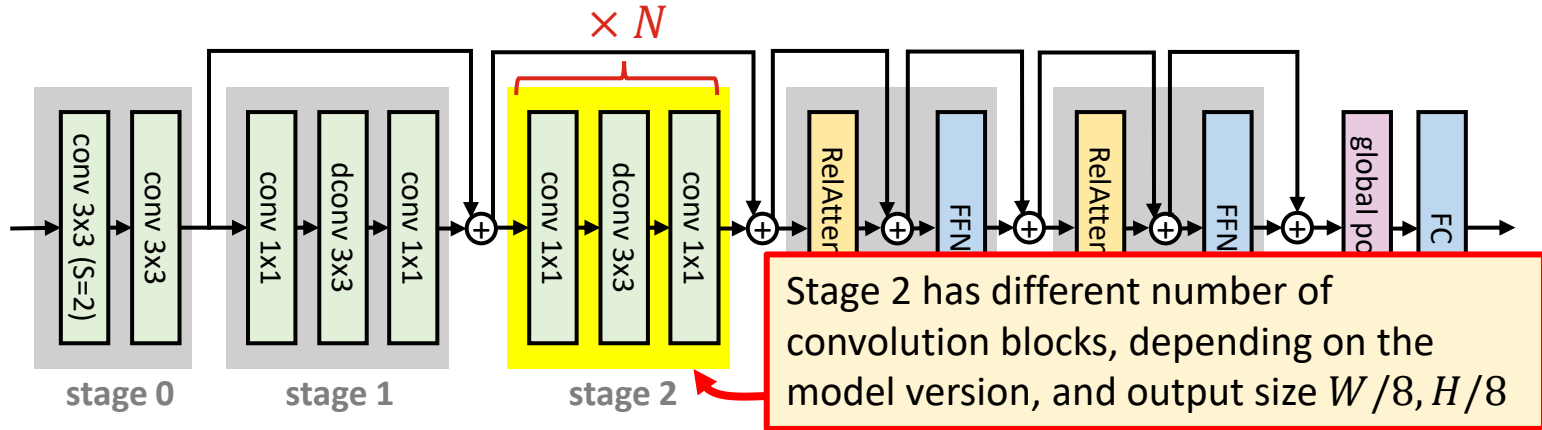
CoAtNet architecture

- Dai et al. (2021) studied different CNN-transformer hybrid configurations and proposed a family of CoAtNet models
 - Three convolution stages and two transformer stages with different number of blocks for different versions of the model



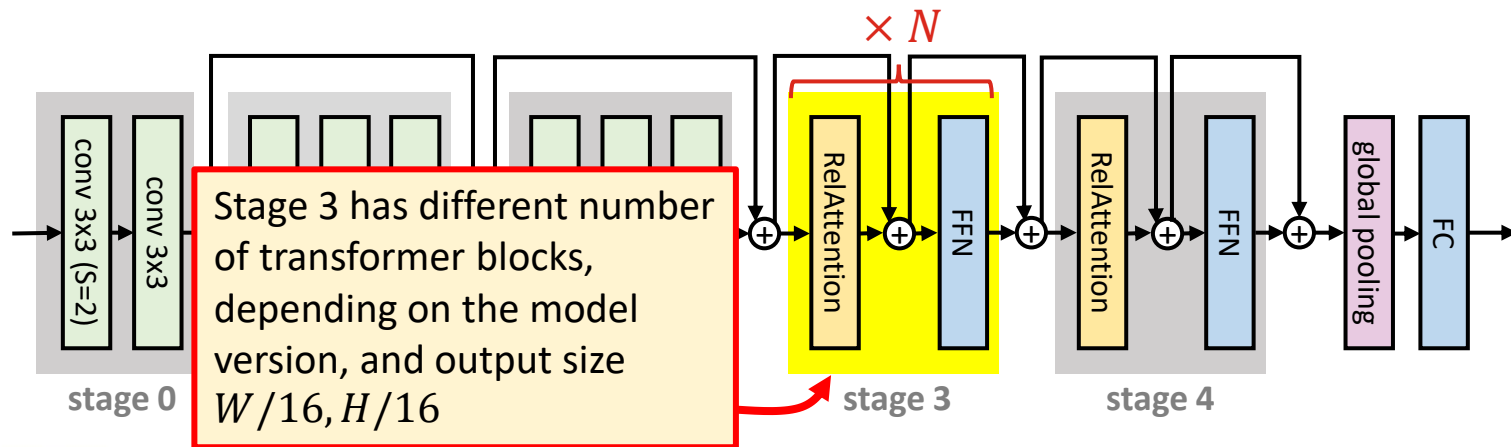
CoAtNet architecture

- Dai et al. (2021) studied different CNN-transformer hybrid configurations and proposed a family of CoAtNet models
 - Three convolution stages and two transformer stages with different number of blocks for different versions of the model



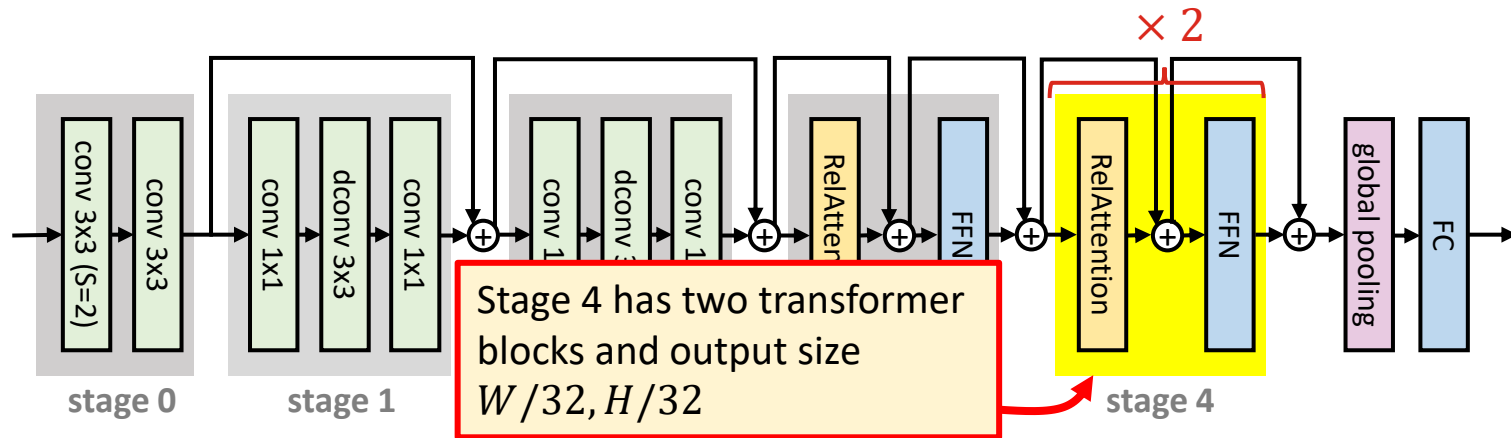
CoAtNet architecture

- Dai et al. (2021) studied different CNN-transformer hybrid configurations and proposed a family of CoAtNet models
 - Three convolution stages and two transformer stages with different number of blocks for different versions of the model



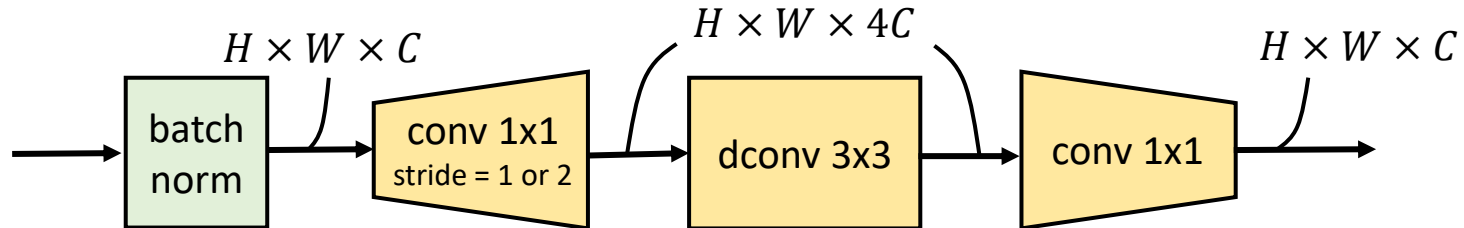
CoAtNet architecture

- Dai et al. (2021) studied different CNN-transformer hybrid configurations and proposed a family of CoAtNet models
 - Three convolution stages and two transformer stages with different number of blocks for different versions of the model



CoAtNet convolution blocks

- The convolution blocks in stages 1 and 2 use an inverted bottleneck, depthwise 3x3 convolutions, and GELU activation function
 - 1x1 point convolution before depthwise convolution (dconv) expands the channel size fourfold, and another one squeezes it back after dconv
 - In the first conv layer, stride is set as 2 for spatial downsampling
 - Skip connections use downsampling by max pooling of stride 2



CoAtNet relative self-attention

- CoAtNet transformer blocks use *relative self-attention* that differs from the self-attention used in original transformer
 - Aims to combine benefits of attention and convolution

- Baseline self-attention weights computed as:

$$y_i = \sum_{j \in \mathcal{G}} \frac{\exp(x_i^T x_j)}{\sum_{k \in \mathcal{G}} \exp(x_i^T x_k)} x_j$$

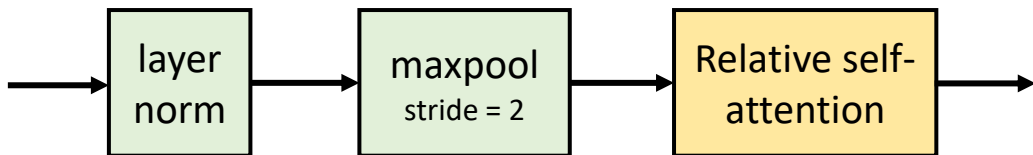
- Relative self-attention computed as:

$$y_i = \sum_{j \in \mathcal{G}} \frac{\exp(x_i^T x_j + k_{i-j})}{\sum_{k \in \mathcal{G}} \exp(x_i^T x_k + k_{i-k})} x_j$$

- Values of k_i are obtained from a learnable matrix that allows to mimic convolution operation

CoAtNet transformer block

- In the first transformer block of each stage, max pooling with stride 2 is used for spatial downsampling
- Note that input is used directly for computing attention, so there is no embedding layer or positional embedding required
 - Global attention is used, but complexity is not prohibitive, since resolution decreased by the convolution stages



CoAtNet architecture comparisons

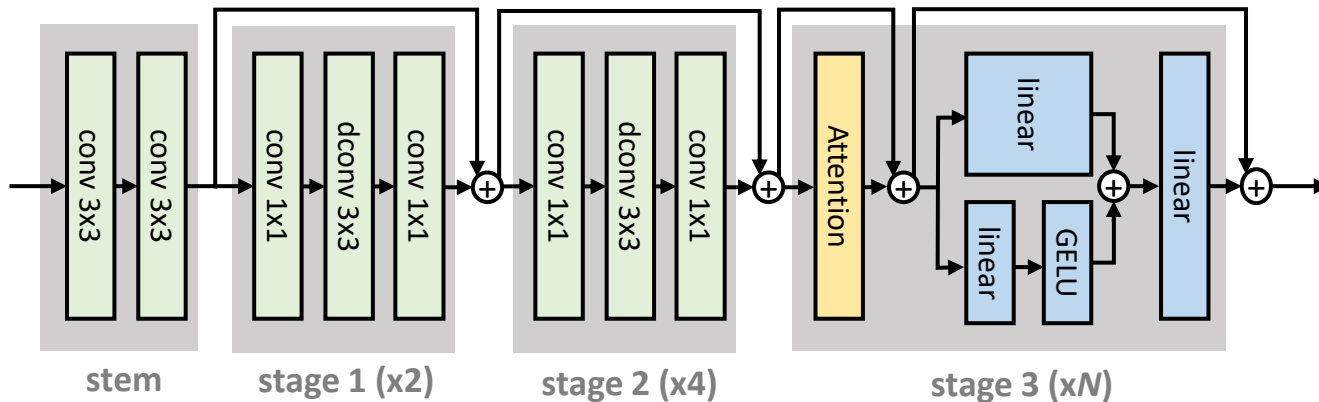
- Capacity of different configurations including different number of convolution and transformer blocks were investigated

Stage 1	Stage 2	Stage 3	Stage 4	Performance
conv	conv	transformer	transformer	best
conv	transformer	transformer	transformer	best
transformer	transformer	transformer	transformer	average
conv	conv	con	transformer	poor
conv	conv	conv	conv	worst

- The conclusion is that in the best model the first one or the first two stages are convolution stages

ViTamin architecture

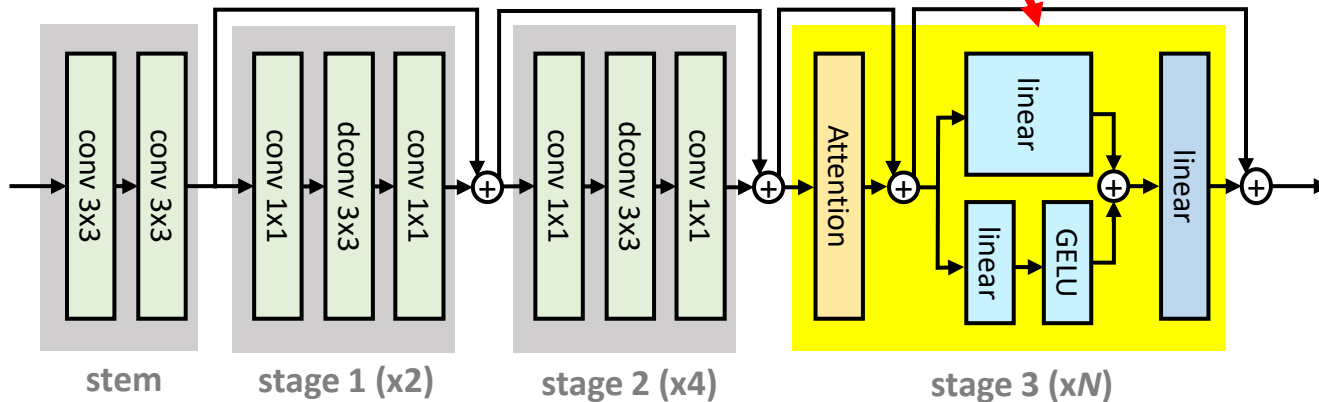
- Chen et al. (2024) proposed ViTamin architecture that is essentially an improved version of CoAtNet
 - Achieves competitive accuracy with lower complexity than visual backbone networks based on full transformer architectures



ViTamin architecture

- Chen et al. (2024) proposed ViTamin architecture that is essentially an improved version of CoAtNet
- Achieves competitive performance with lower complexity than visual backbone networks

The main difference between CoAtNet and ViTamin is different design of the transformer block with gated linear units



Other hybrid CNN-transformer models

- Especially in medical imaging community, several hybrid models with CNN and transformers have been proposed
 - Models for different modalities, including 1D models
 - Both sequential and parallel architectures
- In spite of active research, ViT is currently still the most widely used visual backbone network
 - Alternative architectures show essential performance improvement mostly on specific datasets and applications only
 - The performance bottleneck may be in training strategies and datasets rather than in model architectures?

Summary

- Since different types of neural networks have different strengths and weaknesses, it is often reasonable to use them in combination
 - CNN-RNN hybrid models are useful for analysing sequences of images, including video
 - CNN-transformer hybrid models can benefit from CNNs extracting low level features and transformers processing high level semantic features
- A huge number of hybrid models proposed in research literature
 - Most models are for specific purposes and not widely adopted
 - Some prominent architectures like CoAtNet and ViTamin introduced

References

- CNN-RNN hybrid models
 - Vinyals et al.: *“Show and Tell: A Neural Image Caption Generator,”* CVPR 2015.
 - You et al.: *“Deep Neural Networks for No-Reference Video Quality Assessment,”* ICIP 2019.
 - https://keras.io/examples/vision/video_classification/
- CNN-Transformer hybrid models
 - Carion et al.: *“End-to-end Object Detection with Transformers,”* ECCV 2020.
 - Dai et al.: *“CoAtNet: Marrying Convolution and Attention for All Data Sizes,”* NeurIPS 2021.
 - Chen et al.: *“ViTamin: Designing Scalable Vision Models in the Vision-Language Era,”* CVPR 2024.

Questions and discussion